

PSTAT 231-Assignment3

Xinyi Yang

May 10th 2026

Setup

```
1 library(tidyverse)
2 library(tidymodels)
3 library(corrplot)
4 library(poissonreg)
5 library(discrim) # for LDA and QDA models
6 tidymodels_prefer()
7
8 set.seed(131)
9
10 # Load data; convert survived and pclass to factors
11 df <- read.csv("titanic.csv") %>%
12   mutate(
13     survived = factor(survived, levels = c("Yes", "No")),
14     pclass   = factor(pclass)
15   )
16
17 glimpse(df)
```

Listing 1: Package loading and data import

The Titanic dataset contains 891 observations and 12 variables. `survived` (Yes/No) is the binary outcome. Key predictors include `pclass` (ticket class: 1st, 2nd, 3rd), `sex`, `age`, `sib_sp` (siblings/spouses aboard), `parch` (parents/children aboard), and `fare`. `survived` is relevelled so that "Yes" is the first (positive) level, which is required for `tidymodels` to compute metrics (e.g. AUC) correctly.

Question 1

```
1 data_split <- initial_split(df, prop = 0.75, strata = survived)
2
3 train_data <- training(data_split)
4 test_data  <- testing(data_split)
5
6 dim(df)      # [1] 891  12
```

```

7 dim(train_data) # [1] 667 12
8 dim(test_data)  # [1] 224 12
9
10 train_data %>% summary()

```

Listing 2: Stratified split on survived

We use a 75/25 stratified split: 667 training observations and 224 testing observations (74.9% and 25.1% of the total 891 rows, respectively).

Potential issues noted from `summary()`:

- **age** has **139 missing values** in the training set ($\approx 20.8\%$), which must be handled by imputation in the recipe.
- **cabin** has very many missing values ($\sim 77\%$) and is excluded from the model predictors.
- **embarked** has 2 missing values.
- The training set has 256 survivors (Yes) and 411 non-survivors (No), confirming the class imbalance.

The outcome variable is moderately imbalanced (survivors $\approx 38\%$ vs. non-survivors $\approx 62\%$). Stratified sampling on **survived** ensures that both the training and testing datasets preserve this original proportion. Without stratification, random splitting could produce sets with noticeably different survival rates, leading to biased model training and unreliable evaluation. Stratification thus produces more reliable performance estimates.

Question 2

```

1 train_data %>% ggplot(aes(x = survived)) + geom_bar()
2
3 train_data %>%
4   ggplot(aes(x = survived, fill = sex)) +
5   geom_bar(position = "fill")
6
7 train_data %>%
8   ggplot(aes(x = survived, fill = pclass)) +
9   geom_bar(position = "fill")

```

Listing 3: Percent stacked bar charts

The percent stacked bar chart reveals a dramatic difference in sex composition between survivors and non-survivors. Among passengers who survived, the overwhelming majority are female; among those who did not survive, the majority are male. This reflects the historical “women and children first” evacuation policy. Sex is very likely to be a strong predictor of survival.

Among survivors, first-class passengers represent a much larger share than among non-survivors, where third-class passengers dominate. This indicates that wealthier passengers

in first class had better access to lifeboats or information. Passenger class is likely to be a useful predictor, though perhaps not as strong as sex.

Why percent stacked bar chart over traditional stacked bar chart?

A traditional stacked bar chart shows absolute counts, making it difficult to compare proportions across groups when the total group sizes differ (e.g., there are more non-survivors than survivors). A *percent* stacked bar chart normalises each bar to 100%, so the viewer can directly compare the *relative composition* of each group — which is what matters for assessing whether a predictor is associated with the outcome.

Question 3

```
1 train_data %>%
2   select(where(is.numeric)) %>%
3   select(-passenger_id) %>%
4   cor(use = "complete.obs") %>%
5   corrplot(type = "lower", diag = FALSE)
```

Listing 4: Correlation matrix of continuous predictors

Patterns observed:

The correlation matrix reveals the following patterns among the continuous variables (`age`, `sib_sp`, `parch`, `fare`):

- `sib_sp` and `parch` (moderate positive correlation): passengers with more siblings/spouses also tend to travel with more parents/children — both capture overall family size.
- `fare` and `sib_sp/parch` (weak positive correlations): larger families may pay higher fares, or families tend to book higher-class tickets.
- `age` and `sib_sp` (weak negative correlation): older passengers tend to travel with fewer siblings/spouses.
- Overall, no predictors show a strong correlation with one another, so multicollinearity is not a major concern for these continuous variables.

Question 4

```
1 titanic_recipe <- recipe(survived ~ pclass + sex + age +
2                           sib_sp + parch + fare,
3                           data = train_data) %>%
4   step_impute_linear(age, impute_with = imp_vars(all_predictors()))
5   %>%
6   step_dummy(all_nominal_predictors()) %>%
7   step_interact(term = ~ starts_with("sex"):age + age:fare)
```

Listing 5: Recipe with imputation, dummy coding, and interactions

Notes on recipe design:

- `step_impute_linear(age, impute_with = imp_vars(all_predictors()))` uses all other available predictors to impute missing `age` values via a linear model, preserving relationships between `age` and other covariates.
- `step_dummy()` converts `pclass` (3 levels) and `sex` (2 levels) into dummy variables, dropping one level as the reference category.
- The `sex × age` interaction captures the possibility that age affects survival differently for males and females.
- The `age × fare` interaction captures the possibility that older passengers in higher fare classes had different survival probabilities.

Question 5

```
1 log_model <- logistic_reg() %>%  
2   set_engine("glm") %>%  
3   set_mode("classification")  
4  
5 log_workflow <- workflow() %>%  
6   add_model(log_model) %>%  
7   add_recipe(titanic_recipe)  
8  
9 log_fit <- log_workflow %>% fit(data = train_data)
```

Listing 6: Logistic regression model and workflow

We specify a logistic regression model using the "glm" engine (base R's `glm()` with `family = binomial`). Logistic regression models the log-odds of the positive class ("Yes") as a linear combination of predictors, producing probability outputs in (0, 1).

Question 6

```
1 library(discrim)  
2  
3 lda_model <- discrim_linear() %>%  
4   set_engine("MASS") %>%  
5   set_mode("classification")  
6  
7 lda_workflow <- workflow() %>%  
8   add_model(lda_model) %>%  
9   add_recipe(titanic_recipe)  
10  
11 lda_fit <- lda_workflow %>% fit(data = train_data)
```

Listing 7: LDA model and workflow

Linear Discriminant Analysis assumes that the predictors within each class are multivariate normally distributed with a *common* covariance matrix across classes. It finds linear boundaries in the feature space that best separate the two survival classes.

Question 7

```
1 qda_model <- discrim_quad() %>%  
2   set_engine("MASS") %>%  
3   set_mode("classification")  
4  
5 qda_workflow <- workflow() %>%  
6   add_model(qda_model) %>%  
7   add_recipe(titanic_recipe)  
8  
9 qda_fit <- qda_workflow %>% fit(data = train_data)
```

Listing 8: QDA model and workflow

Quadratic Discriminant Analysis is similar to LDA but relaxes the assumption of a common covariance matrix, allowing each class to have its *own* covariance structure. This results in quadratic (curved) decision boundaries, giving QDA more flexibility than LDA at the cost of estimating more parameters.

Question 8

```
1 knn_model <- nearest_neighbor(neighbors = 10) %>%  
2   set_engine("kkn") %>%  
3   set_mode("classification")  
4  
5 knn_workflow <- workflow() %>%  
6   add_model(knn_model) %>%  
7   add_recipe(titanic_recipe)  
8  
9 knn_fit <- knn_workflow %>% fit(data = train_data)
```

Listing 9: KNN classification model ($k = 10$) and workflow

We choose $k = 10$ neighbours. KNN classifies a new observation by majority vote among its 10 nearest neighbours (measured by Euclidean distance in the feature space). It is a non-parametric method with no distributional assumptions.

Question 9

```
1 library(yardstick)  
2  
3 log_preds <- predict(log_fit, train_data, type = "prob") %>%  
4   bind_cols(train_data)
```

```

5 roc_auc(log_preds, truth = survived, .pred_Yes)
6 ## # A tibble: 1 x 3
7 ##   .metric .estimator .estimate
8 ##   <chr>   <chr>         <dbl>
9 ## 1 roc_auc binary      0.869
10
11 lda_preds <- predict(lda_fit, train_data, type = "prob") %>%
12   bind_cols(train_data)
13 roc_auc(lda_preds, truth = survived, .pred_Yes)
14 ## # A tibble: 1 x 3
15 ##   .metric .estimator .estimate
16 ##   <chr>   <chr>         <dbl>
17 ## 1 roc_auc binary      0.865
18
19 qda_preds <- predict(qda_fit, train_data, type = "prob") %>%
20   bind_cols(train_data)
21 roc_auc(qda_preds, truth = survived, .pred_Yes)
22 ## # A tibble: 1 x 3
23 ##   .metric .estimator .estimate
24 ##   <chr>   <chr>         <dbl>
25 ## 1 roc_auc binary      0.855
26
27 knn_preds <- predict(knn_fit, train_data, type = "prob") %>%
28   bind_cols(train_data)
29 roc_auc(knn_preds, truth = survived, .pred_Yes)
30 ## # A tibble: 1 x 3
31 ##   .metric .estimator .estimate
32 ##   <chr>   <chr>         <dbl>
33 ## 1 roc_auc binary      0.966
34
35 # Summary table
36 results <- tibble(
37   Model = c("Logistic", "LDA", "QDA", "kNN"),
38   AUC    = c(0.869, 0.865, 0.855, 0.966)
39 )
40 results
41 ## # A tibble: 4 x 2
42 ##   Model      AUC
43 ##   <chr>   <dbl>
44 ## 1 Logistic 0.869
45 ## 2 LDA      0.865
46 ## 3 QDA      0.855
47 ## 4 kNN      0.966

```

Listing 10: Generate probability predictions and compute AUC on training data

Training AUC Results

Model	Training AUC
Logistic Regression	0.869
LDA	0.865
QDA	0.855
KNN ($k = 10$)	0.966

KNN achieves the highest training AUC (0.966) by a large margin. This is expected: with $k = 10$, KNN can memorise local structure in the training data very closely, which does not necessarily generalise well to unseen data.

Question 10

```
1 log_test_preds <- predict(log_fit, test_data, type = "prob") %>%
2   bind_cols(test_data)
3 log_test_auc <- roc_auc(log_test_preds, truth = survived, .pred_Yes)
4
5 lda_test_preds <- predict(lda_fit, test_data, type = "prob") %>%
6   bind_cols(test_data)
7 lda_test_auc <- roc_auc(lda_test_preds, truth = survived, .pred_Yes)
8
9 qda_test_preds <- predict(qda_fit, test_data, type = "prob") %>%
10  bind_cols(test_data)
11 qda_test_auc <- roc_auc(qda_test_preds, truth = survived, .pred_Yes)
12
13 knn_test_preds <- predict(knn_fit, test_data, type = "prob") %>%
14  bind_cols(test_data)
15 knn_test_auc <- roc_auc(knn_test_preds, truth = survived, .pred_Yes)
16
17 test_results <- tibble(
18   Model      = c("Logistic", "LDA", "QDA", "kNN"),
19   Testing_AUC = c(log_test_auc$.estimate, lda_test_auc$.estimate,
20                  qda_test_auc$.estimate, knn_test_auc$.estimate)
21 )
22 test_results %>% arrange(desc(Testing_AUC))
23
24 ## # A tibble: 4 x 2
25 ##   Model      Testing_AUC
26 ##   <chr>          <dbl>
27 ## 1 kNN            0.851
28 ## 2 Logistic      0.848
29 ## 3 LDA           0.843
30 ## 4 QDA           0.831
```

Listing 11: Evaluate all four models on testing data

Testing AUC Results

Model	Train AUC	Test AUC
Logistic Regression	0.869	0.848
LDA	0.865	0.843
QDA	0.855	0.831
KNN ($k = 10$)	0.966	0.851

Best model on testing data: KNN ($k = 10$) with Test AUC = 0.851.

Confusion Matrix and ROC Curve (KNN, Best Model)

```
1 best_class_preds <- predict(knn_fit, test_data, type = "class") %>%
2   bind_cols(test_data)
3
4 cm <- conf_mat(best_class_preds, truth = survived, estimate = .pred_
5   class)
6 cm
7 ## Truth
8 ## Prediction Yes No
9 ##           Yes 63 18
10 ##           No 23 120
11
12 autoplot(cm, type = "heatmap")
13
14 best_prob_preds <- predict(knn_fit, test_data, type = "prob") %>%
15   bind_cols(test_data)
16
17 roc_data <- roc_curve(best_prob_preds, truth = survived, .pred_Yes)
18 autoplot(roc_data)
```

Listing 12: Confusion matrix and ROC curve for KNN (best model)

Confusion matrix interpretation:

	Truth: Yes	Truth: No
Predicted: Yes	63 (TP)	18 (FP)
Predicted: No	23 (FN)	120 (TN)

The model correctly classifies 63 survivors and 120 non-survivors. It produces 18 false positives (predicted survived but did not) and 23 false negatives (predicted did not survive but actually did). Overall accuracy = $(63 + 120)/224 = 81.7\%$.

Model Comparison and Discussion

How did the best model perform?

The KNN model ($k = 10$) achieved a training AUC of 0.966 and a testing AUC of 0.851, showing a large gap of 0.115 between training and testing performance. This gap indicates that the KNN model is overfitting the training data.

Why does this gap exist?

KNN is a flexible, non-parametric model. When k is relatively small, it can fit the training data very closely by adapting to local patterns. However, some of these patterns are specific to the training sample (noise) and do not generalise to unseen data, causing the testing AUC to drop substantially. Despite this overfitting, KNN still achieves the highest testing AUC (0.851) among all four models — but the gap suggests that a larger k or cross-validated tuning could improve its generalisation further.

The three parametric models (Logistic, LDA, QDA) show much smaller train-test gaps, confirming that their stronger assumptions (linearity, normality) act as regularisers that prevent overfitting.

Question 11

The inverse of the logistic function $p(z) = \frac{e^z}{1 + e^z}$ is the logit function $z(p) = \ln\left(\frac{p}{1-p}\right)$.

Proof:

We wish to find the inverse function: given $p = \frac{e^z}{1 + e^z}$, solve for z in terms of p .

Note that

$$1 - p = 1 - \frac{e^z}{1 + e^z} = \frac{1 + e^z - e^z}{1 + e^z} = \frac{1}{1 + e^z}.$$

Form the odds ratio:

$$\frac{p}{1-p} = \frac{\frac{e^z}{1+e^z}}{\frac{1}{1+e^z}} = \frac{e^z}{1+e^z} \cdot (1+e^z) = e^z.$$

Take the natural logarithm of both sides:

$$\ln\left(\frac{p}{1-p}\right) = \ln(e^z) = z.$$

Therefore, $z(p) = \ln\left(\frac{p}{1-p}\right)$, which is exactly the logit function.

The logit function maps $p \in (0, 1)$ to $z \in (-\infty, +\infty)$, and the logistic function maps back. They are mutual inverses: $p(z(p)) = p$ and $z(p(z)) = z$.

Question 12

Effect of Increasing x_1 by 2

Let $z = \beta_0 + \beta_1 x_1$. The odds of the outcome are:

$$\text{odds}(x_1) = \frac{p}{1-p} = e^z = e^{\beta_0 + \beta_1 x_1}.$$

When x_1 increases by 2 (to $x_1 + 2$), the new odds are:

$$\text{odds}(x_1 + 2) = e^{\beta_0 + \beta_1(x_1 + 2)} = e^{\beta_0 + \beta_1 x_1 + 2\beta_1} = e^{\beta_0 + \beta_1 x_1} \cdot e^{2\beta_1} = \text{odds}(x_1) \cdot e^{2\beta_1}.$$

Conclusion: Increasing x_1 by 2 multiplies the odds by a factor of $e^{2\beta_1}$. In general, a one-unit increase in x_1 multiplies the odds by e^{β_1} ; a two-unit increase multiplies the odds by $e^{2\beta_1}$.

- If $\beta_1 > 0$: $e^{2\beta_1} > 1$, so the odds *increase*.
- If $\beta_1 < 0$: $e^{2\beta_1} < 1$, so the odds *decrease*.
- If $\beta_1 = 0$: $e^{2\beta_1} = 1$, so the odds are *unchanged*.

Limiting Behaviour When $\beta_1 < 0$

Assume $\beta_1 < 0$. Then as $x_1 \rightarrow \infty$:

$$z = \beta_0 + \beta_1 x_1 \rightarrow -\infty \quad (\text{since } \beta_1 < 0).$$

Therefore:

$$p = \frac{e^z}{1 + e^z} \xrightarrow{z \rightarrow -\infty} \frac{e^{-\infty}}{1 + e^{-\infty}} = \frac{0}{1 + 0} = 0.$$

As $x_1 \rightarrow -\infty$:

$$z = \beta_0 + \beta_1 x_1 \rightarrow +\infty \quad (\text{since } \beta_1 < 0 \text{ and } x_1 \rightarrow -\infty).$$

Therefore:

$$p = \frac{e^z}{1 + e^z} = \frac{1}{e^{-z} + 1} \xrightarrow{z \rightarrow +\infty} \frac{1}{0 + 1} = 1.$$

When $\beta_1 < 0$, x_1 has a *negative* effect on the probability of class 1. As x_1 grows very large, $p \rightarrow 0$ (the outcome becomes increasingly unlikely). As x_1 becomes very negative, $p \rightarrow 1$ (the outcome becomes near certain). This is the expected monotone decreasing behaviour of the logistic function with a negative coefficient.